



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Data Types

CMSC 124 Problem Set 1

Prepared by: Rene Andre Bedonia Jocsing

Published: September 25, 2026

Deadline: October 06, 2026

Welcome to CMSC 123 in Nightmare Mode™.

Unit 5 of the syllabus, chapter 6 of Sebesta's *Concepts of Programming Languages*, is a list of 10 kinds of data: integers, strings, enumerations, arrays, associative arrays, records, tuples, lists, unions, and pointers. Read about them and they blur together (they did while I was reading it). A paragraph explaining that an associative array stores its keys while an ordinary array doesn't is a paragraph you'll nod while reading and forget by Thursday.

So we're not going to read about them or lecture about them. That's boring. Lectures are boring. Active learning's the coolest thing in the block these days. So, instead of listening to an instructor yap, you're going to **build** one of each data type, in C17, over the next 10 days, with a partner. That's one data type a day, reasonable since CMSC 123 is the course's prerequisite.

Why C, you might be *moaning* right now? C is the right language for this precisely because it's so unhelpful. It gives you the raw mechanisms: signed arithmetic whose overflow it refuses to define, array expressions that readily become bounds-free pointers, null-terminated strings, and a free union that doesn't remember which member is active. You have to construct the fences before you can say "it just works." And once you've written the bounds check yourself, "Java checks array bounds" stops being a sentence you read and becomes a convenience whose machinery you can see.

In hindsight, Assembly was the better choice.

Suppose an array starts at index -1 and holds 3 elements. Index 2 is one past its end. Built-in C array access won't stop you from reading there. The result has undefined behavior, which means C doesn't prescribe one outcome. Maybe you get plausible data. Maybe the process stops. C has left the room.

Read Background and The Case File Language before you write code. Keep the matching module section and `include/dt.h` open while you work. Reread Common Pitfalls before your final check.

Background

A Type Is a Set of Values and the Operations on Them

That's the whole definition. Sebesta's first line of chapter 6 defines a data type as a collection of values and a set of operations on those values. An unsigned 8-bit integer is the values 0 through 255 together with addition, subtraction, and the rest. Change either half and you have a different type. This is why `int` and `float` are different types even on a machine where both are 32 bits. Same storage, different values, but `+` means something different in each.

Every design question in Unit 5 is a question about one of those two halves. What values are in the set? What happens when an operation would produce a value that isn't?

Two pieces of the starter's interface encode that split before you write any code. Every operation that can fail returns a `dt_status`, one code per way an operation can fail:

```

1  typedef enum {
2      DT_OK = 0,
3      DT_ERR_OVERFLOW, /* a checked integer operation left the usable range */
4      DT_ERR_RANGE,    /* an index or an ordinal fell outside its bounds */
5      DT_ERR_KEY,      /* an associative array has no such key */
6      DT_ERR_FIELD,    /* a record has no field with that name */
7      DT_ERR_TAG,      /* a value was read as the wrong alternative */
8      DT_ERR_EMPTY,    /* car or cdr was taken of the empty list */
9      DT_ERR_CAPACITY, /* a requested allocation or representation is unavailable */
10     DT_ERR_RELEASED,  /* a reference was read or released after release */
11     DT_ERR_LEAK       /* a reference was still holding memory when the program ended */
12 } dt_status;

```

And every value carries a `dt_tag` saying which alternative is live:

```

1  typedef enum {
2      DT_NIL = 0,
3      DT_INT,
4      DT_STR,
5      DT_ENUM,
6      DT_ARRAY,
7      DT_MAP,
8      DT_RECORD,
9      DT_TUPLE,
10     DT_LIST,
11     DT_REF
12 } dt_tag;

```

You don't write either enum. You just write the code that uses them. The status codes are how the driver tells you what went wrong. It prints the message that goes with a status (from `dt_status_message`) and exits, so your modules never print anything themselves.

Each code belongs to a small set of functions. Read the table once now, then again when a module section tells you which code an operation owes you:

Code	Returned by	When
DT_OK	every function that returns a <code>dt_status</code>	the operation succeeded
DT_ERR_OVERFLOW	<code>dt_int_add</code> , <code>dt_int_sub</code> , <code>dt_int_mul</code>	the result would leave the range of long long
DT_ERR_RANGE	<code>dt_str_substr</code> , <code>dt_enum_name</code> , <code>dt_enum_from_name</code> , <code>dt_array_get</code> , <code>dt_array_set</code> , <code>dt_map_key_at</code> , <code>dt_record_field_name</code> , <code>dt_tuple_at</code>	an index, an ordinal, or a spelling fell outside the domain it was checked against
DT_ERR_KEY	<code>dt_map_get</code> , <code>dt_map_remove</code>	the map has no such key. An absent key is a different answer from nil
DT_ERR_FIELD	<code>dt_record_get</code> , <code>dt_record_set</code>	the record declares no field with that name, and it won't grow one

Code	Returned by	When
DT_ERR_TAG	dt_value_as_int, dt_value_as_enum, dt_value_as_str	the tag says the value holds a different alternative
DT_ERR_EMPTY	dt_list_car, dt_list_cdr	the list is empty, so there's no first cell to read
DT_ERR_CAPACITY	dt_str_append, dt_str_substr, dt_map_put, and the driver whenever a constructor returns NULL	an allocation failed, a fixed limit was exceeded, or a required size or index range can't be represented
DT_ERR_RELEASED	dt_ref_borrow, dt_ref_release	the reference had already been released
DT_ERR_LEAK	the driver's sweep at exit	a reference still held its cell when the program ended

Constructors are the exception to the first rule. They return a pointer and report failure with `NULL`. The driver turns that `NULL` into `DT_ERR_CAPACITY`.

A Refresher in C

The header keeps repeating a small set of C forms. Take `dt_value *out`. The `*` says `out` points to a `dt_value`, and `*out = value` writes into the caller's object. That's how a function returns a status and still hands back a value. You'll use this pattern everywhere.

A struct groups fields. A typedef gives the structure a shorter type label. For a structure value `v`, `v.tag` selects a field. For a pointer `p`, `p->tag` is shorthand for `(*p).tag`.

A pointer stores an address. `NULL` means it points to no object. A `const` parameter promises that the function won't change the object through that pointer.

The type `size_t` is an unsigned integer type that can represent an object's size. `sizeof(dt_value)` gives that size in bytes, while `sizeof *out` gives the size of the object that `out` points to. The second form stays correct if the pointed-to type changes.

Three library functions manage dynamic storage. For a positive `n`, `malloc(n)` requests `n` bytes and returns `NULL` if the request fails. Avoid a zero-byte request here because C permits either a null pointer or a special pointer for it. `realloc(p, n)` resizes an allocation and may move it. If a positive-size request fails, the old allocation remains valid, so keep the result in a temporary pointer until you've checked it. `free(p)` releases an allocation, and `free(NULL)` does nothing. Each successful allocation in these modules needs one matching release by its owner.

The Ten Categories Are Not Ten Separate Things

Look at what your interface has to hold and one structure keeps showing up.

An array element can be an integer or a string. A record field can be an integer or a string. A list holds whatever you *cons* onto it. Every container in Unit 5 has the same problem. It needs one slot type that can hold any of your language's values. It also needs a way to ask a value what it currently is.

That's the union, the ninth category on the list. It lets one value slot carry any of the other categories. So the interface has exactly one value type, with a member for every category except itself:

```

1  typedef struct {
2      dt_tag tag;
3      union {
4          long long integer; /* DT_INT */
5          int ordinal; /* DT_ENUM */
6          dt_str *string; /* DT_STR */

```

```

7      dt_array  *array;    /* DT_ARRAY */
8      dt_map    *map;      /* DT_MAP   */
9      dt_record *record;   /* DT_RECORD */
10     dt_tuple  *tuple;    /* DT_TUPLE */
11     dt_list   *list;     /* DT_LIST  */
12     dt_ref    *ref;      /* DT_REF   */
13     } as;
14 } dt_value;

```

A tag saying which alternative is live, with a union holding the alternatives. Every container stores `dt_value`. You build this once and spend the rest of the 10 days consuming it. One alternative is the empty value, `nil`, built by `dt_value_nil()` and written `nil` in case files. It's a value like any other, with its own tag. Several modules use it to say "nothing is here." Failures still travel through the status code. That distinction comes up again in the list and map sections.

If that design looks familiar, it should. It's the same decision Laboratory Activity 3 asks you to make for your interpreter's runtime values. A Rust group reaching for an `enum` or a C++ group reaching for `std::variant` is reaching for this. Their compiler maintains the tag. Here you maintain it by hand, which teaches you what the compiler was doing for you.

The Tag Is Only Good If You Check It

C's union is a **free union**. It stores one of several alternatives and keeps no record of which. Nothing stops this.

```

1  dt_value v = dt_value_int(42);
2  dt_str *s = v.as.string;      /* the bits of 42, read as an address */

```

The compiler is fine with it. There's no run-time error. What happens next depends on whether the address 42 happens to be readable. That's machine-dependent.

A **discriminated union**, the kind you're building, pairs the union with a tag. This assignment's checked readers compare that tag before they read a member. Languages with built-in sum types, such as Rust, ML, and Swift, let the compiler enforce more of that relationship than C does here.

Your `dt_value_as_int` is where you write that check by hand.

Two Things You Get to Not Worry About (Phew)

Ownership, mostly. Each module owns its representation storage. For example, an array owns its descriptor and element block. A container borrows any objects named by its `dt_value` elements. The driver owns those runtime objects and frees them when the program ends. If an array holds a string, freeing the array must leave the string alone. The `dt_ref` module then isolates one explicit ownership exercise. **Parsing**, entirely. The case files that drive your library are read by a front end you're given, complete. You never write a parser here.

Learning Objectives

By the end of this problem set, you should be able to:

- Explain what a type is in terms of its values and its operations, and use that framing to predict how a language will behave at a boundary
- Detect signed integer overflow before it happens, and say why detecting it afterward doesn't work in C
- Represent a string in a way that makes its length a field read rather than a walk, and say what that buys and what it costs

- Implement an array descriptor with an arbitrary lower bound, and write the bounds check that a language without one doesn't need
 - Distinguish an array from an associative array by which one stores its selectors, and implement both
 - Distinguish a record from a tuple by whether its parts are named, and a list from an array by which end is cheap
 - Implement a discriminated union whose readers check the tag, and explain what a free union can't promise
 - Detect access after release, a double release, and an unreleased allocation, and explain how the driver reports each one
-

The Pair

Pairs for this problem set are assigned by me, at random, drawn across every section. You don't choose your partner and you probably won't get your laboratory partner. If the pooled roster across all sections is odd, one group of three is formed and gets the same work as everyone else.

The [pairing sheet](#) is authoritative. If it disagrees with your memory of what somebody said in class, the sheet wins.

This is the lecture component, so none of the laboratory group policy applies here. There's no progress report, booking, or defense appointment. The Milestones section offers an advisory schedule. Nobody collects daily reports. The deadline itself is strict.

Two reasons explain why the pairs are random and cross-section. You'll read code written by someone whose habits you didn't grow up with, which is most of what code review turns out to be. Splitting the 10 modules between 2 people means each of you has to explain your half to somebody who didn't write it, in a window short enough that you can't put the conversation off.

You won't always be free at the same hours. Nobody expects that. This is asynchronous work. The repository is where you meet. Push small commits, describe what you did in the message, and open an issue on your own fork when you're stuck. That records the question for your partner before tomorrow. If the two of you can't find any overlapping time, tell me in the first few days. That's a problem I can solve early and can't solve the night before the deadline.

Setup

You need a C compiler, CMake, Ninja, and Python 3.9 or newer. These are the same tools the C++ track in Laboratory Activity 0 installs, so if your group took that track, you already have all of it and can skip this section.

Windows. Install MSYS2 from PowerShell, then work in the **MSYS2 UCRT64** terminal from the Start menu. The plain MSYS2 shell and Git Bash will both give you a different compiler:

```
1 winget install --id MSYS2.MSYS2 --exact
```

```
1 pacman -Syu
2 pacman -S --needed git curl mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-cmake
   ↪ mingw-w64-ucrt-x86_64-ninja mingw-w64-ucrt-x86_64-python
```

If `pacman -Syu` tells you to close the terminal, close it, reopen UCRT64, and run it again. Run every command in this manual from that UCRT64 terminal.

Linux (Debian or Ubuntu).

```

1 sudo apt update
2 sudo apt install -y build-essential cmake ninja-build python3 git curl

```

macOS.

You need Homebrew for the second command. If `brew --version` fails, install Homebrew first, then open a new terminal.

```

1 xcode-select --install
2 brew install cmake ninja python

```

Verify.

```

1 gcc --version
2 cmake --version
3 ninja --version
4 python3 --version

```

Then get the starter, which contains the header, the driver, the stubs you'll fill in, and every test that grades you. Fork [cmssc-124-ps1-starter](#) on GitHub and clone your fork. A fork is a snapshot: a fix I push to the starter mid-run never reaches it, so what you were graded against can't change under you.

```

1 git clone https://github.com/<your-account>/cmssc-124-ps1-starter.git
2 cd cmssc-124-ps1-starter
3 ./build.sh
4 ./check.sh

```

One of you forks the repository under your account, keeps the fork public, and adds the other member as a collaborator, since the fork is the pair's deliverable. Then open the Actions tab on the fork and enable workflows. GitHub disables them on every fork. At first, that tab shows a short notice with an enable button. GitHub documents the rule and the fix under [Workflows in forked repositories](#). The workflow run is a deliverable, so don't discover this on Day 10.

The build should go smoothly. `check.sh` should fail, loudly, with 33 of 63 checks passing. That's the correct starting state. Reading [Your First Failing Run](#) explains why 33 of them pass when nothing has been written yet.

The Starter Repository

The starter is a small C project. Here's the layout you'll deal with:

1	include/dt.h	the fixed public interface. Do not edit.
2	src/main.c	given, complete
3	src/driver.c	given, complete: the case-file front end
4	src/driver.h	given, complete: the driver's internals. Do not edit.
5	src/print.c	given, complete: the canonical printed forms
6	src/dt_*.c	yours, 10 files
7	cases/	the entire grading corpus
8	ANALYSIS.md	yours, create this joint analysis

```

9  README.md           yours, fill the Pair section only
10 CMakeLists.txt       the build definition. Do not edit.
11 .github/workflows/test.yml your CI run of the same checks
12 build.sh run check.sh the course run contract

```

Three scripts run the project. `./build.sh` compiles once into `build/`. `./run <case-file>` executes one case file through the driver. `./check.sh` builds, runs the whole corpus, then runs it again under sanitizers. A 63/63 result completes the public automated gate. It isn't your final grade. The rubric separately assesses your analysis, collaboration history, and memory evidence.

Two rules from the header run through the whole interface. The Background section has already shown you their machinery.

1. **Every operation that can fail returns `dt_status`.** None of them report failure with a special return value. There's no integer `dt_int_add` could return that wouldn't also be a correct answer. When an operation fails, the driver prints the matching message and exits 70.
2. **Modules own representation storage, not borrowed values.** An array owns its descriptor and element block. It doesn't own objects held through the `dt_value` elements in that block. The driver's environment owns those runtime objects. `dt_ref` owns its private cell and releases it explicitly.

You may change only the 10 `src/dt_*.c` files, `ANALYSIS.md`, and the Pair section in `README.md`. Everything else in the repository is complete and fixed. Email `reflection.txt` separately. Don't add it to the repository.

The Task

Implement the 10 files named `src/dt_*.c`. Create `ANALYSIS.md` with your joint answers. Fill the Pair section in `README.md` with every member's full name and GitHub username. Those are the only repository changes you may make. Email `reflection.txt` separately. Don't add it to the repository.

`include/dt.h` is fixed. `src/main.c`, `src/driver.c`, `src/driver.h`, and `src/print.c` are given to you complete. `cases/` is the grading corpus and it's already in your repository. If you find yourself editing any of those, you've wandered off the assignment.

Reading Your First Failing Run

A fresh clone passes 33 of 63 checks. That number is misleading. Day one is a better time to learn how than day 10.

The corpus is 63 case files under `cases/`. Ten live under `normal/` and need working code. The rest check boundaries and refusals. Most of the passing 33 are cases that expect a refusal. `boundary/array_index_above_upper` wants exit 70, and a `dt_array_get` that returns `DT_ERR_RANGE` no matter what it's asked is exit 70. It passes. It has also done nothing. A stub that refuses everything accidentally satisfies every test whose subject is a refusal.

The 10 cases under `normal/` all fail. Those are the ones that need working code. Watch them first. When all 10 pass, most of the rest follow.

The Case File Language

Your library is driven by case files. Here's one:

```

1  # cases/normal/array_basics.case
2  arr new a 3 0

```

```

3 arr len a
4 arr set a 0 10
5 arr set a 1 20
6 arr set a 2 30
7 arr get a 1
8 print @a

```

which produces:

```

1 3
2 20
3 [10, 20, 30]

```

Every line is one command. A # starts a comment. Commands that read something print one line. Commands that build or modify something print nothing. An identifier binds an object, and @name refers to a bound object.

Value Literals

Anywhere a command takes a value, it accepts one of 5 things:

Written	Means
42, -7	an integer
"hello"	a string
enum:RED	an enumeration value
nil	the nil value
@name	whatever name is bound to

Strings take the escapes \", \\, \n, \t, \0, and \xNN. The \0 is there so a case file can build a string with a zero byte in the middle, which is the case that separates your string type from C's.

The Commands

```

1 int add A B          int sub A B          int mul A B
2
3 str new NAME "text"  str append NAME "text"  str len NAME
4 str substr NAME START LEN OUT      str eq NAME NAME
5
6 enum of NAME         enum name ORDINAL
7
8 arr new NAME LEN LOWER  arr set NAME INDEX VALUE
9 arr get NAME INDEX      arr len NAME
10
11 map new NAME          map put NAME "key" VALUE
12 map get NAME "key"     map del NAME "key"      map len NAME
13
14 rec new NAME FIELD...  rec set NAME FIELD VALUE  rec get NAME FIELD
15
16 tup new NAME VALUE...  tup at NAME INDEX        tup arity NAME
17
18 list nil NAME          list cons NAME VALUE TAIL
19 list car NAME           list cdr NAME OUT      list len NAME
20
21 ref new NAME VALUE     ref borrow NAME           ref release NAME
22

```

23	<code>as int VALUE</code>	<code>as str VALUE</code>	<code>as enum VALUE</code>
24	<code>tag VALUE</code>	<code>print VALUE</code>	

`str substr` and `list cdr` take an extra trailing identifier because they produce a new object that needs somewhere to live.

`enum of` and the field positions of `rec new` and `rec set` take a bare word. That word can spell an enumerator such as `RED` or identify a field. It carries no `@`, so it doesn't refer to a bound object. It also carries no `enum:` prefix, so it isn't a value literal. Anywhere else a command takes a value, all 5 forms from Value Literals work.

How Values Print

The corpus compares stdout byte for byte, so every value has exactly one spelling. You don't implement any of this. `src/print.c` does. You do need to recognise it when you're reading a diff.

Kind	Prints as
integer	42
string	"hello", with the escapes above
enumeration	RED
nil	nil
array	[10, 20, 30]
associative array	{"alpha" -> 1, "beta" -> 2} in insertion order
record	{name = "ada", age = 36} in declaration order
tuple	(1, "two")
list	(1 2 3), and () when empty
reference	ref(42), or ref(released)

`str eq` prints true or false. It's the only command whose output isn't a value.

`tag VALUE` prints the tag's spelling, using the same spellings as the table's kinds: `int`, `str`, `enum`, `array`, `map`, `record`, `tuple`, `list`, `ref`, and `nil`. It's a quick way to ask a value what it currently is, which is the question the tagged union exists to answer.

The Interface

10 modules. Each one is one category from the syllabus. Each one poses one design question you should be able to answer out loud by the deadline. When a function has an `out` parameter, it writes through that pointer only on `DT_OK`. Every failure leaves the caller's earlier value untouched.

Checked Integers, `src/dt_int.c`

A checked integer is the machine's `long long` together with a rule for what happens when an operation would produce a value the type can't hold. Checked integers are useful when a program must report an impossible result before C evaluates it.

Checked integer arithmetic

The range check happens before C evaluates the arithmetic.

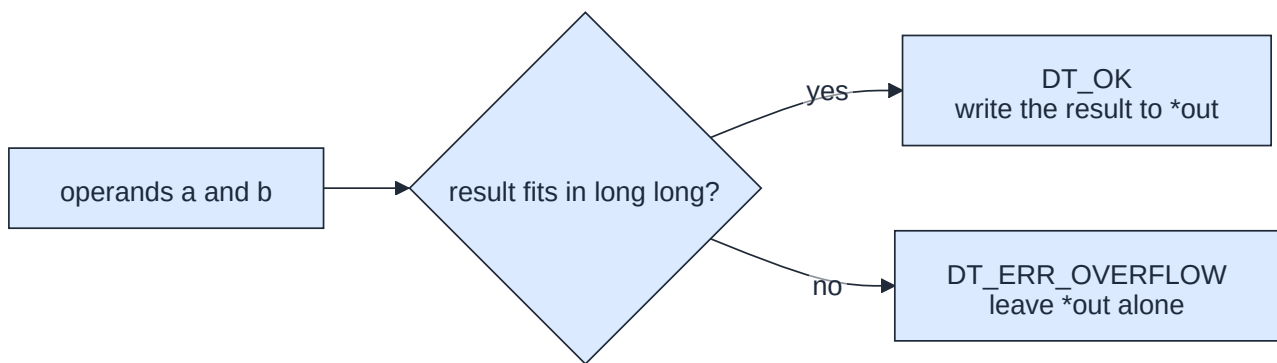
Check before computing

Figure 1: Check before computing

Start with `cases/normal/int_arithmetic.case`. Then use the `int_*` files under `cases/boundary/` to drive each overflow refusal.

```

1 dt_status dt_int_add(long long a, long long b, long long *out);
2 dt_status dt_int_sub(long long a, long long b, long long *out);
3 dt_status dt_int_mul(long long a, long long b, long long *out);

```

All 3 write their result through `out` and report through the return value. On failure `*out` is left exactly as the caller had it:

Function	What it does	Fails with	Why that code
<code>dt_int_add</code>	writes <code>a + b</code> to <code>*out</code>	<code>DT_ERR_OVERFLOW</code>	the sum leaves the range of <code>long long</code> , and no integer could stand in for that failure without also being a possible correct answer
<code>dt_int_sub</code>	writes <code>a - b</code> to <code>*out</code>	<code>DT_ERR_OVERFLOW</code>	the same range, with <code>b = LLONG_MIN</code> as the case that negating <code>b</code> can't reach
<code>dt_int_mul</code>	writes <code>a * b</code> to <code>*out</code>	<code>DT_ERR_OVERFLOW</code>	the same range, with <code>LLONG_MIN * -1</code> as the case a division-based check trips over

Signed overflow in C is **undefined behavior**, which is a stronger and stranger claim than “the result wraps.” It means the standard places no requirement on what the program does at all, so the compiler is entitled to optimize on the assumption that overflow never occurs. That has a consequence people find out about the hard way:

```

1 long long sum = a + b;
2 if (b > 0 && sum < a) return DT_ERR_OVERFLOW; /* may be deleted */

```

When b is positive, $\text{sum} < a$ can happen only if the addition overflowed. But the overflow has already happened by the time the condition runs. C lets an optimizer assume that a defined program never reaches it, so the guarded branch may disappear. `build.sh` builds Release, so your own build is an optimizing one. The check has to run *before* the arithmetic, phrased in operands that can't overflow themselves. If b is positive, overflow means $a > \text{LLONG_MAX} - b$. That right-hand side is always computable.

Multiplication has more cases. One of them is the reason a division-based check needs care: $\text{LLONG_MIN} / -1$ overflows too, because LLONG_MIN has no positive counterpart. Two's complement has one more negative value than positive values. That asymmetry is where it bites.

Run that check on 2 inputs and watch it decide. For `dt_int_add(LLONG_MAX, 1, &out)`, b is 1, so the question is whether $a > \text{LLONG_MAX} - 1$. That right-hand side is computable, LLONG_MAX is bigger than it, and the answer is `DT_ERR_OVERFLOW` with `out` never written. For `dt_int_add(2, 3, &out)`, the same question reads $2 > \text{LLONG_MAX} - 3$. That's false, so the addition runs and `out` becomes 5.

The design question here is what a language should do on overflow. C defines unsigned arithmetic modulo one more than the maximum value, while signed overflow has undefined behavior. Python integers grow as needed. Rust checks according to the build profile and overflow-check settings, so code may panic or use wrapping behavior. Ada can raise an exception. What's your take?

Strings, `src/dt_str.c`

A string is a sequence of characters or bytes treated as one value. Programs use strings for text, identifiers, file contents, and any other data whose length isn't known when the program is compiled. This assignment's string also keeps embedded zero bytes, so its stored length matters more than a terminator.

A string with its own descriptor

The handle records where the bytes are, how many are data, and how much room exists.

Keep length beside the bytes

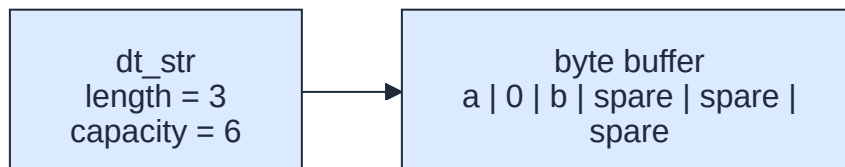


Figure 2: Keep length beside the bytes

Start with `cases/normal/string_building.case`. Then run the substring boundary cases. Use this case for embedded zero bytes:

```
1 cases/capacity/embedded_zero_byte.case
```

It catches the shortcuts that ordinary text doesn't expose.

```

1 dt_str  *dt_str_new(const char *bytes, size_t length);
2 void    dt_str_free(dt_str *s);
3 size_t  dt_str_len(const dt_str *s);
4 const char *dt_str_bytes(const dt_str *s);
5 dt_status dt_str_append(dt_str *s, const char *bytes, size_t length);
6 dt_status dt_str_substr(const dt_str *s, size_t start, size_t length, dt_str **out);
7 bool    dt_str_eq(const dt_str *a, const dt_str *b);
  
```

Two of these can fail. The rest either answer a question or release memory, so they have nothing to report:

Function	What it does	Fails with	Why that code
<code>dt_str_new</code>	copies <code>length</code> bytes into a new string	returns <code>NULL</code>	a constructor returns a pointer, so it has no status to carry a failure. The driver turns <code>NULL</code> into <code>DT_ERR_CAPACITY</code>
<code>dt_str_free</code>	releases the buffer and the handle	cannot fail	a <code>NULL</code> argument is accepted and ignored
<code>dt_str_len</code>	reads the stored length	cannot fail	the length is a field, so nothing about reading it can go wrong
<code>dt_str_bytes</code>	hands back the byte buffer	cannot fail	the implementation may expose an internal terminator, but callers must read exactly <code>dt_str_len</code> data bytes
<code>dt_str_append</code>	adds bytes to the end, growing the buffer when they don't fit	<code>DT_ERR_CAPACITY</code>	the allocation failed or the required size can't include the terminator. The string keeps its earlier bytes
<code>dt_str_substr</code>	builds a new string from <code>length</code> bytes starting at <code>start</code>	<code>DT_ERR_RANGE</code> or <code>DT_ERR_CAPACITY</code>	<code>RANGE</code> when the requested piece runs past the end, <code>CAPACITY</code> when the new string can't be allocated
<code>dt_str_eq</code>	true when both hold the same bytes	cannot fail	it answers a question rather than performing an operation

A C string is a null-terminated character sequence stored in an array. An array expression often converts to a pointer to its first element. That pointer doesn't carry the array's length or capacity. `strlen` walks to the first zero byte, so a loop that repeatedly calls it can become quadratic when the loop also traverses the string. An embedded zero ends the C string even if later bytes remain in the array.

Your `dt_str` stores a length and capacity beside its byte buffer. Length then costs one field read, embedded zero bytes remain data, and `append` can determine whether the allocation is large enough. The representation costs extra fields and still requires careful growth. One corpus case stores the 3 data bytes 'a', zero, and 'b'. That difference is the point.

Grow the buffer geometrically when an `append` doesn't fit. If you instead grow by only the required amount on every small `append`, repeated copying can make a long `append` sequence quadratic.

The terminator needs one byte even though it isn't part of the stored length. Check that `length + 1` is representable before a constructor allocates it. For `append`, check how much room remains before adding the old and new lengths. Doing the addition first can wrap `size_t` and turn an impossible request into a small one.

`dt_str_substr` is the one whose failure needs care, so work it out on a string `s` holding `hello`, whose length is 5.

```
1 dt_str_substr(s, 3, 2, &out) -> DT_OK,      *out is "lo"
```

```
2 dt_str_substr(s, 5, 0, &out) -> DT_OK,          *out is ""
3 dt_str_substr(s, 3, 5, &out) -> DT_ERR_RANGE,    *out untouched
```

The second line asks for nothing, starting one position past the last byte, and that’s a legal request whose answer is the empty string. The third asks for five bytes when only 2 remain. Write the test in 2 steps. Compare `start` against the `length` first, then compare `length` against what’s left after `start`. Adding them together and comparing the sum can wrap around, because `size_t` is unsigned, and a wrapped sum compares as a small number that passes the check.

Enumerations, `src/dt_enum.c`

An enumeration gives readable labels to a small set of choices. It’s useful when a value must be one of a known set, such as a color, a direction, or a parser state. The **enumerator** is the C identifier, such as `DT_COLOR_GREEN`. Its **ordinal** is its integer position. Green has ordinal 1. Its **spelling** is the text "GREEN" that appears in a case file and in printed output.

Three views of one enumeration value

The C identifier, stored integer, and case-file text all refer to green.

Map the value in both directions

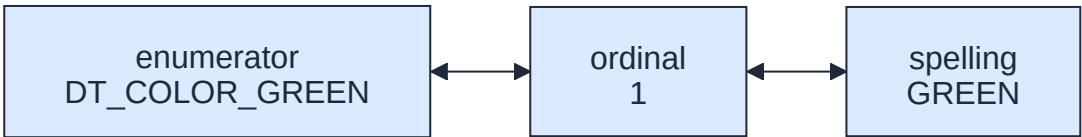


Figure 3: Map the value in both directions

`cases/normal/enum_names.case` exercises all 3 operations and both directions of the mapping.

```
1 typedef enum {
2     DT_COLOR_RED = 0,
3     DT_COLOR_GREEN,
4     DT_COLOR_BLUE,
5     DT_COLOR_COUNT
6 } dt_color;
7
8 bool dt_enum_is_valid(int ordinal);
9 dt_status dt_enum_name(int ordinal, const char **out);
10 dt_status dt_enum_from_name(const char *name, int *out);
```

Function	What it does	Fails with	Why that code
dt_enum_is_valid	true when the ordinal is one of the 3 values	cannot fail	it's the question the other 2 ask before they act

Function	What it does	Fails with	Why that code
<code>dt_enum_name</code>	writes the value's spelling to <code>*out</code>	<code>DT_ERR_RANGE</code>	an ordinal outside 0 through <code>DT_COLOR_COUNT - 1</code> isn't in the domain, so there's no spelling to hand back
<code>dt_enum_from_name</code>	writes the ordinal for that spelling to <code>*out</code>	<code>DT_ERR_RANGE</code>	a word that isn't one of the 3 isn't in the domain either, and there's no numeric fallback

C permits an enumeration object to hold values that no declared enumerator uses. Assigning 47 can therefore produce a value outside this module's color domain. Every later read has to cope with that value. Languages differ in how they restrict conversions between integers and enumeration values. This module accepts only the 3 declared color ordinals.

You can't change what C's `enum` does. You can put the domain check in the one place every read goes through. These 3 functions are that place. `DT_COLOR_COUNT` records the size of the domain, and `dt_enum_is_valid` checks an ordinal against it.

`dt_enum_from_name` runs the same domain check from the other side. It walks the 3 spellings, comparing with `strcmp`, and writes the position of the one that matches.

```

1 dt_enum_from_name("GREEN", &out)  -> DT_OK,          out = 1
2 dt_enum_from_name("PURPLE", &out) -> DT_ERR_RANGE, out untouched
3 dt_enum_from_name("1", &out)      -> DT_ERR_RANGE, out untouched

```

Read the last line twice. "1" spells an ordinal that exists. It still fails. A spelling isn't a number. Reaching for `atoi` here would put back the coercion this module exists to remove.

Arrays, `src/dt_array.c`

An array maps a consecutive range of integer indices to a fixed block of elements. Arrays are useful when position carries meaning and constant-time indexed access matters. In this module, `lower_bound` is the first legal index. An **offset** is the zero-based position in the element block. The array stores its length and lower bound in a **run-time descriptor**, the metadata its operations need while the program runs.

Logical array indices and storage offsets

The descriptor maps an arbitrary lower bound onto zero-based storage.

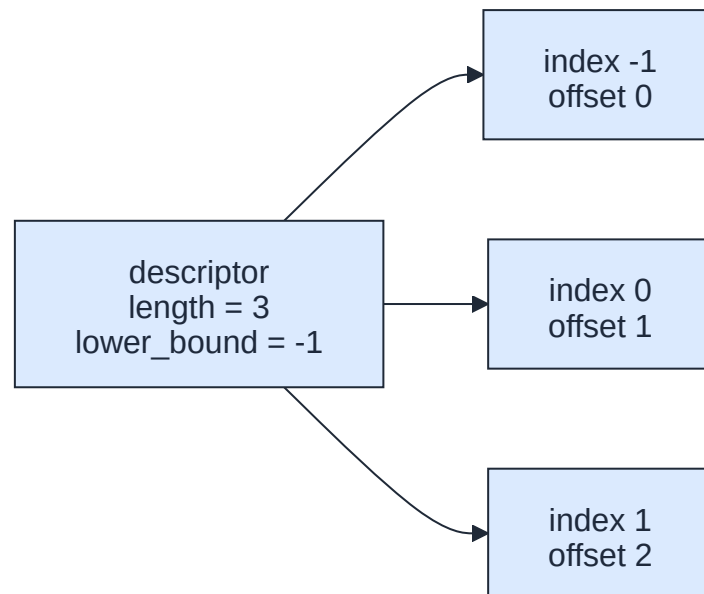
Translate an index to an offset

Figure 4: Translate an index to an offset

Start with `cases/normal/array_basics.case`. The `array_*` boundary and capacity cases then exercise both ends of the index range and both size overflows.

```

1 dt_array *dt_array_new(size_t length, long long lower_bound);
2 void      dt_array_free(dt_array *a);
3 size_t    dt_array_len(const dt_array *a);
4 long long dt_array_lower_bound(const dt_array *a);
5 dt_status dt_array_get(const dt_array *a, long long index, dt_value *out);
6 dt_status dt_array_set(dt_array *a, long long index, dt_value v);
  
```

Function	What it does	Fails with	Why that code
<code>dt_array_new</code>	builds length nil elements whose indices start at <code>lower_bound</code>	returns NULL	the allocation size or final index is unrepresentable, allocation fails, or another capacity limit is reached. A length of 0 is legal
<code>dt_array_free</code>	releases the element block and the descriptor	cannot fail	the values inside belong to the environment and are left alone
<code>dt_array_len</code>	reads the stored length	cannot fail	the length is a field, and it doesn't depend on the lower bound
<code>dt_array_lower_bound</code>	reads the stored lower bound	cannot fail	this is the number get and set subtract

Function	What it does	Fails with	Why that code
<code>dt_array_get</code>	writes the element at index to <code>*out</code>	<code>DT_ERR_RANGE</code>	the nonnegative distance from the lower bound is at least the stored length
<code>dt_array_set</code>	replaces that element with <code>v</code>	<code>DT_ERR_RANGE</code>	the same check, and nothing changes when it fires

Built-in C array offsets start at 0. That convention hides a step. Ada, Fortran, and Pascal let you declare an array over `1..10` or `-5..5`. The mapping is:

```
1 offset = index - lower_bound
```

The access function reads the lower bound from the descriptor. One corpus case builds an array over `-1..1`. Another builds one over `1..3`.

The bounds check has 2 ends. Forgetting the upper end is the famous mistake. Forgetting the lower end is less famous. A below-bound index can address storage outside the allocation. Accessing it has undefined behavior.

Write the check once as a helper and call it from both `get` and `set`. Keep two copies of a bounds check and they drift. The copy that drifts is the one you forgot to test.

Run that helper over an array of 3 elements built with `dt_array_new(3, -1)`, so its indices are `-1, 0`, and `1`.

```
1 dt_array_get(a, -1, &out) -> DT_OK,      offset -1 - (-1) = 0
2 dt_array_get(a,  1, &out) -> DT_OK,      offset  1 - (-1) = 2
3 dt_array_get(a,  2, &out) -> DT_ERR_RANGE, distance 3 equals the length
4 dt_array_get(a, -2, &out) -> DT_ERR_RANGE, -2 is below the lower bound
```

Never evaluate `index - lower_bound` as signed `long long` across the full range. For `LLONG_MAX - LLONG_MIN`, the mathematical answer doesn't fit. First reject an index below the lower bound. Then compute the nonnegative distance with unsigned arithmetic and compare it with `length`. The constructor must also reject a nonempty array whose final index can't fit in `long long`. Reject the array when `length * sizeof(dt_value)` can't fit in `size_t`, too.

Associative Arrays, `src/dt_map.c`

An associative array, or map, binds stored keys to values. It's useful when a selector is a word such as "beta". A **hash** turns a key into a number. That number selects a **bucket**, one small part of the table. Two keys can select the same bucket, which is a **collision**. **Chaining** handles collisions by linking the entries in each bucket and comparing their stored keys during lookup.

A hash map with stable traversal order

Buckets serve lookup while a separate sequence preserves insertion order.

Keep lookup and traversal structures

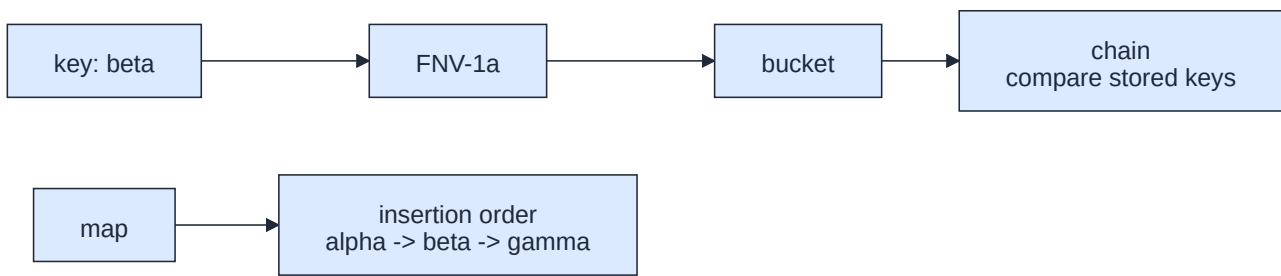


Figure 5: Keep lookup and traversal structures

Start with `cases/normal/map_basics.case`. Use `cases/cleanup/map_churn.case` after removal works, since it repeats the allocation and unlinking path.

```

1 dt_map *dt_map_new(void);
2 void dt_map_free(dt_map *m);
3 size_t dt_map_len(const dt_map *m);
4 dt_status dt_map_put(dt_map *m, const char *key, dt_value v);
5 dt_status dt_map_get(const dt_map *m, const char *key, dt_value *out);
6 dt_status dt_map_remove(dt_map *m, const char *key);
7 dt_status dt_map_key_at(const dt_map *m, size_t index, const char **out);
  
```

Function	What it does	Fails with	Why that code
<code>dt_map_new</code>	builds an empty map	returns NULL	a constructor has no status to return. An empty map is a real object
<code>dt_map_free</code>	releases the nodes, the copied keys, and the map	cannot fail	the values belong to the environment
<code>dt_map_len</code>	how many keys are bound right now	cannot fail	replacing an existing key's value doesn't change it
<code>dt_map_put</code>	binds <code>v</code> to a copy of <code>key</code>	<code>DT_ERR_CAPACITY</code>	the only failure is an allocation. A <code>put</code> can't fail for a missing key, since adding a missing key is what it does
<code>dt_map_get</code>	writes the bound value to <code>*out</code>	<code>DT_ERR_KEY</code>	the key is absent, which is a different answer from a key bound to <code>nil</code>
<code>dt_map_remove</code>	unlinks the key from its bucket and from the order	<code>DT_ERR_KEY</code>	removing a key that was never there is a loud refusal rather than a plain success
<code>dt_map_key_at</code>	writes the key at that position in insertion order	<code>DT_ERR_RANGE</code>	an index at or past the key count is outside the order

An array omits its indices. This map stores its keys. An array position uses arithmetic. A map lookup uses the hash to choose a bucket, then compares the keys along that bucket's chain. An empty bucket needs no comparison.

Use the 64-bit FNV-1a hash. Its core is short enough to see all at once:

```

1 unsigned long long h = 14695981039346656037ULL;
2 for (const unsigned char *p = (const unsigned char *)key; *p != '\0'; p++) {
3     h ^= (unsigned long long)*p;
4     h *= 1099511628211ULL;
5 }

```

The first constant is the starting accumulator. The ULL suffix makes each constant an unsigned long long, the unsigned type the reference uses for the running hash. Casting key to const unsigned char * gives every input byte a value from 0 through 255, even on a compiler where plain char is signed. The ^ operator exclusive-ors that byte into the accumulator. The multiplication changes the accumulator again before the next byte. Unsigned overflow is defined to wrap, so each pass keeps the low bits and never invokes signed overflow. Finally, take the hash modulo your bucket count to obtain a valid bucket index.

Map printing has to follow insertion order, or nobody can write an expected-output file. Bucket order is an artifact of the hash function. So keep a separate record of keys in insertion order. Putting an existing key again replaces its value and leaves its position alone. Removing a key takes it out of that order. Putting it back later appends it at the end. A corpus case runs that sequence and checks the printed order.

dt_map_put is where all of that meets, so walk one call through it. Suppose the map already holds alpha, beta, and gamma, in that order, and the call is dt_map_put(m, "beta", dt_value_int(22)).

```

1 1. hash "beta"          -> a bucket number
2 2. find that bucket     -> a chain of entries, possibly several
3 3. search the chain for "beta" -> found
4 4. overwrite its value with 22 -> DT_OK

```

Nothing else moves. The insertion list still reads alpha, beta, gamma, so the map still prints in that order. Now run the same 4 steps for dt_map_put(m, "delta", ...). Step 3 searches the chain and finds nothing, so step 4 needs 2 appends. Put a new entry on the bucket's chain. Then put the copied key at the end of the insertion list. Copy the key in that step. The caller's buffer belongs to the command that's running and won't outlive it.

Python's dict preserved insertion order as an implementation detail in 3.6 and as a language guarantee in 3.7. Python also keeps OrderedDict, a dict subclass with methods specialized for rearranging dictionary order. Your map implements the smaller promise the printer needs: iteration follows insertion order.

Records, src/dt_record.c

A record groups a fixed set of fields under one value. Each field has its own spelling and value. Records are useful for entities such as a person, where name and age explain their contents better than positions 0 and 1. The record's fixed collection of declared fields is its **schema**.

A record with a fixed schema

Each declared field spelling and value occupy matching positions.

Align fields with values

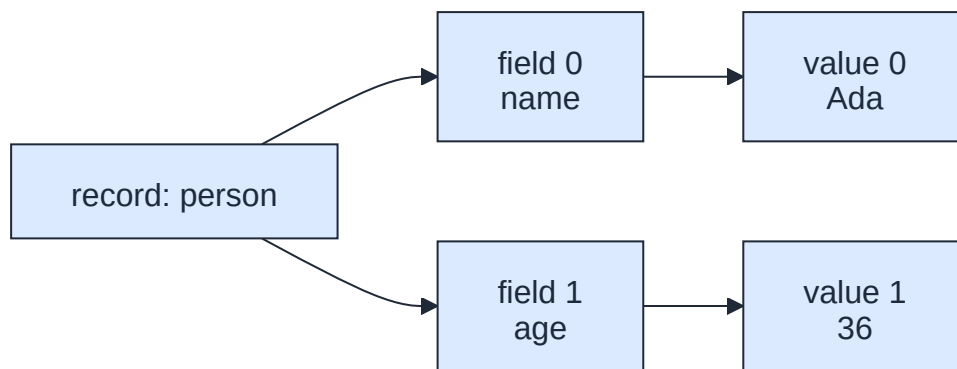


Figure 6: Align fields with values

Start with `cases/normal/record_basics.case`. Then use `cases/boundary/record_unknown_field.case` and the 2 record capacity cases to check the fixed schema.

```

1  #define DT_RECORD_MAX_FIELDS 8
2
3  dt_record *dt_record_new(const char **field_names, size_t field_count);
4  void      dt_record_free(dt_record *r);
5  size_t    dt_record_field_count(const dt_record *r);
6  dt_status dt_record_field_name(const dt_record *r, size_t index, const char **out);
7  dt_status dt_record_get(const dt_record *r, const char *field, dt_value *out);
8  dt_status dt_record_set(dt_record *r, const char *field, dt_value v);
  
```

Function	What it does	Fails with	Why that code
<code>dt_record_new</code>	builds a record with those field names, every field nil	returns NULL	more than 8 fields or a failed allocation. The driver reports <code>DT_ERR_CAPACITY</code>
<code>dt_record_free</code>	releases the copied names and the record	cannot fail	the field values belong to the environment
<code>dt_record_field_count</code>	how many fields the record has	cannot fail	a record never grows one
<code>dt_record_field_name</code>	the name at that position in declaration order	<code>DT_ERR_RANGE</code>	an index at or past the field count is outside the declaration
<code>dt_record_get</code>	writes that field's value to <code>*out</code>	<code>DT_ERR_FIELD</code>	the record never declared a field with that name
<code>dt_record_set</code>	replaces that field's value	<code>DT_ERR_FIELD</code>	the same reason, and this refusal is the line between a record and a map

In many statically compiled languages, the compiler resolves a declared field to a storage offset. An expression such as `employee.salary` then needs no run-time text search. This assignment makes the

lookup visible. One array holds copied field spellings. A parallel array holds their values. A matching index connects each field to its value.

The field count is capped at `DT_RECORD_MAX_FIELDS`, 8. Ask for more and `dt_record_new` returns `NULL`, which the driver reports as `DT_ERR_CAPACITY`.

An identifier the record never declared produces `DT_ERR_FIELD`. It doesn't become a new field. That refusal is the line between a record and an associative array. In this assignment, it also separates the fixed record schema from a map that can add and remove keys.

Take a record built by `rec new person name age` and walk one lookup. `dt_record_get(r, "age", &out)` compares "age" against `names[0]`, which is "name", and then against `names[1]`, which matches at index 1. The value lives at `values[1]`, so that's what goes into `*out`, and the answer is `DT_OK`. Now try `dt_record_get(r, "salary", &out)`. The same walk reaches the end of the table with no match, so the answer is `DT_ERR_FIELD` and `*out` is untouched. Compare that with `dt_map_get`, where the same miss is `DT_ERR_KEY`, and with `dt_map_put`, which creates the key on the spot.

Tuples, `src/dt_tuple.c`

A tuple groups a fixed number of values and selects them by position. The number of positions is its **arity**. Tuples are useful for short bundles whose positions already have an agreed meaning, such as a coordinate or a function that returns two related results.

A tuple selected by position

Arity records how many fixed positions the tuple contains.

Read a fixed position

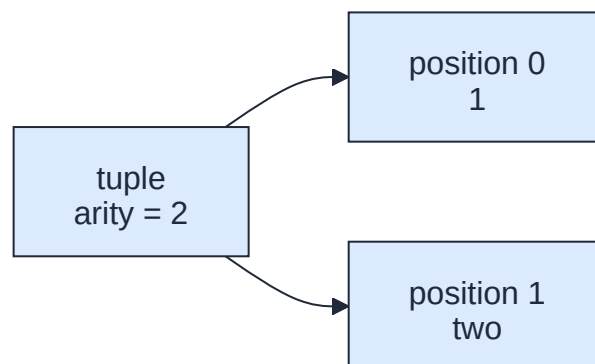


Figure 7: Read a fixed position

Start with `cases/normal/tuple_basics.case`. The tuple boundary and capacity cases distinguish an empty tuple, the maximum arity, and one position too many.

```

1  #define DT_TUPLE_MAX_ARITY 8
2
3  dt_tuple *dt_tuple_new(const dt_value *values, size_t count);
4  void      dt_tuple_free(dt_tuple *t);
5  size_t    dt_tuple_arity(const dt_tuple *t);
6  dt_status dt_tuple_at(const dt_tuple *t, size_t index, dt_value *out);
  
```

Function	What it does	Fails with	Why that code
<code>dt_tuple_new</code>	builds a tuple holding the first <code>count</code> values, in order	returns <code>NULL</code>	more than <code>DT_TUPLE_MAX_ARITY</code> values, or a failed allocation. A count of 0 is the legal empty tuple
<code>dt_tuple_free</code>	releases the tuple	cannot fail	the values inside belong to the environment
<code>dt_tuple_arity</code>	how many parts the tuple has	cannot fail	the arity is fixed when the tuple is built
<code>dt_tuple_at</code>	writes the part at index to <code>*out</code>	<code>DT_ERR_RANGE</code>	positions run from 0 to <code>arity - 1</code> , so the arity itself is already past the end

A tuple selects its parts by position, while this assignment's record selects fields by spelling. `person.age` says what it holds while `person[1]` stays vague. A tuple needs no field declarations.

There's no `dt_tuple_set`. Arity and contents are fixed at construction, capped at `DT_TUPLE_MAX_ARITY`, 8. Python tuples also refuse replacement of a position after construction.

For the tuple `(1, "two")`, whose arity is 2:

```

1 dt_tuple_at(t, 0, &out) -> DT_OK,      *out is the integer 1
2 dt_tuple_at(t, 1, &out) -> DT_OK,      *out is the string "two"
3 dt_tuple_at(t, 2, &out) -> DT_ERR_RANGE, *out untouched

```

Index 2 is the arity. The last legal position is one below it. An empty tuple has arity 0, so every index fails and `dt_tuple_at` never reads the array at all.

Lists, `src/dt_list.c`

A linked list is either empty or one **cell** containing a `head` value and a `tail` pointer to the rest of the list. `cons` adds a cell at the front, `car` reads its head, and `cdr` reads its tail. Linked lists are useful when a program frequently adds at the front or lets several lists share the same tail.

Two lists sharing one tail

Consing a new head reuses the existing tail instead of copying it.

Share the tail cells

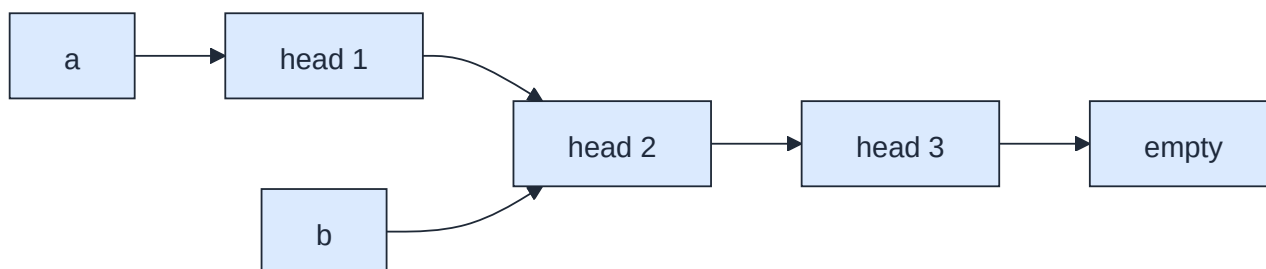


Figure 8: Share the tail cells

Start with `cases/normal/list_basics.case`. Then run the 2 empty-list boundary cases. Use this case for a shared tail:

```
1 cases/cleanup/shared_list_tail.case
```

It checks the sharing decision the normal case can't settle.

```
1 dt_list *dt_list_nil(void);
2 dt_list *dt_list_cons(dt_value head, dt_list *tail);
3 void dt_list_free(dt_list *l);
4 size_t dt_list_len(const dt_list *l);
5 dt_status dt_list_car(const dt_list *l, dt_value *out);
6 dt_status dt_list_cdr(const dt_list *l, dt_list **out);
```

Function	What it does	Fails with	Why that code
<code>dt_list_nil</code>	the empty list	cannot fail	NULL represents empty, so this allocates nothing
<code>dt_list_cons</code>	one new cell holding head and pointing at tail	returns NULL	allocation is its only failure. It shares tail
<code>dt_list_free</code>	releases this one cell	cannot fail	other lists may still reach tail
<code>dt_list_len</code>	counts the cells	cannot fail	one step per cell
<code>dt_list_car</code>	writes the first cell's value to *out	DT_ERR_EMPTY	the empty list has no first cell. Returning nil would report a value where there's an absence
<code>dt_list_cdr</code>	writes the shared tail to *out	DT_ERR_EMPTY	the same reason. The cdr of a one-element list is the empty list, which is a real answer

`dt_list_cons` allocates one cell and points it at the existing tail. It doesn't copy that tail. After this case file:

```
1 list nil e
2 list cons c 3 e
3 list cons b 2 c
4 list cons a 1 b
```

the list a is (1 2 3) and the list b is (2 3). The cell holding 3 is one cell that both of them reach. Consing is constant time and costs one cell no matter how long the tail is. This representation is useful when you frequently add at the front or share tails. Indexed access remains linear.

It's also why `dt_list_free` frees one cell and never follows the tail. Following it would free cells that b still reaches. The corpus builds this arrangement so the sanitizer can catch you if you do.

The empty list is a null pointer, so it costs nothing and needs no allocation. `car` of the empty list is `DT_ERR_EMPTY`. A value that doesn't exist and a value that's nil are different answers.

Run `dt_list_cdr` over the arrangement above.

```

1 dt_list_cdr(a, &out)    -> DT_OK,      *out is b, the same cells, not a copy
2 dt_list_cdr(b, &out)    -> DT_OK,      *out is c
3 dt_list_cdr(c, &out)    -> DT_OK,      *out is the empty list
4 dt_list_cdr(nil, &out)  -> DT_ERR_EMPTY, *out untouched

```

The third line and the fourth are the pair to keep apart. The cdr of a one-element list is an empty list you can keep using. The cdr of an empty list is a question with no answer.

The Union, src/dt_value.c

A union lets several alternatives occupy the same storage. It's useful when one slot must hold different kinds of value at different times. A free C union doesn't record which alternative is active. A **discriminated union** pairs the shared payload with a tag, or discriminator, that identifies the valid alternative before a reader touches it.

A discriminated union

The tag determines which interpretation of the shared payload is valid.

Check the tag before the payload

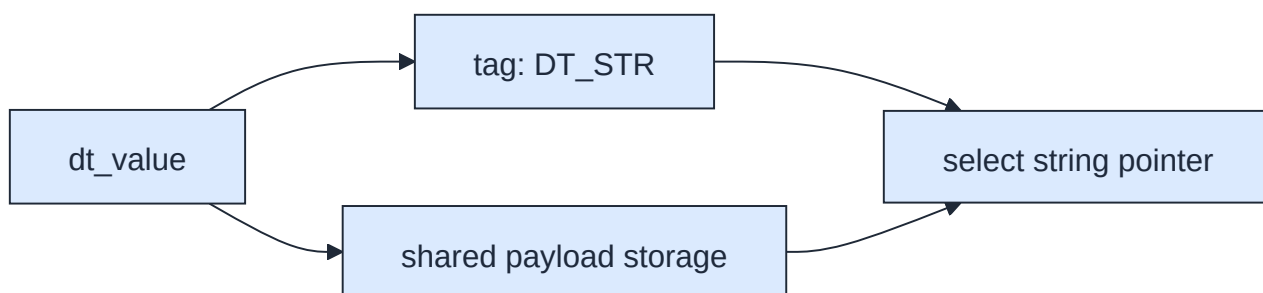


Figure 9: Check the tag before the payload

Start with cases/normal/union_readers.case. The cases under cases/tag/ then try the same reads with the wrong alternative active.

```

1 dt_value dt_value_nil(void);
2 dt_value dt_value_int(long long n);
3 dt_value dt_value_enum(int ordinal);
4 dt_value dt_value_str(dt_str *s);
5 dt_value dt_value_array(dt_array *a);
6 dt_value dt_value_map(dt_map *m);
7 dt_value dt_value_record(dt_record *r);
8 dt_value dt_value_tuple(dt_tuple *t);
9 dt_value dt_value_list(dt_list *l);
10 dt_value dt_value_ref(dt_ref *p);
11
12 dt_status dt_value_as_int(dt_value v, long long *out);
13 dt_status dt_value_as_enum(dt_value v, int *out);
14 dt_status dt_value_as_str(dt_value v, dt_str **out);

```

Function	What it does	Fails with	Why that code
the ten constructors	build a <code>dt_value</code> carrying the matching tag	cannot fail	they only fill in a struct, and they're given to you complete
<code>dt_value_as_int</code>	writes <code>v.as.integer</code> to <code>*out</code>	<code>DT_ERR_TAG</code>	the tag says the value holds a different alternative, and reading anyway is undefined behavior rather than a wrong number
<code>dt_value_as_enum</code>	writes <code>v.as.ordinal</code> to <code>*out</code>	<code>DT_ERR_TAG</code>	the same check on a different alternative
<code>dt_value_as_str</code>	writes <code>v.as.string</code> to <code>*out</code>	<code>DT_ERR_TAG</code>	the same check, and this is the reader where a missed check hands the printer an integer where an address belongs

The constructors are given to you. Each one builds a value with the right tag, from `dt_value_nil` for the empty value up to `dt_value_ref` for a reference. The 3 readers are yours. Each is about 4 lines: check the tag, return `DT_ERR_TAG` if it's wrong, otherwise write the payload.

Resist the shortcut of returning the payload without looking. `as_str 42` would then hand the printer an integer wearing a pointer's clothes. What happens after that has undefined behavior. A sanitizer can report the invalid access. Without one, the program might terminate, print plausible output, or behave differently after a compiler change.

One call passes and one refuses:

```

1 dt_value_as_str(dt_value_str(s), &out) -> DT_OK,      *out is s
2 dt_value_as_str(dt_value_int(42), &out) -> DT_ERR_TAG, *out untouched

```

The second line is the whole module. `v.as.string` and `v.as.integer` occupy the same bytes, so a reader that skips the tag test finds 42 sitting where a `dt_str *` belongs and hands it back as one. The test is `v.tag != DT_STR`. Run it before you read the payload.

Owned References, `src/dt_ref.c`

A reference reaches a value indirectly through another storage cell. References are useful when code needs to share access or manage a lifetime separately from the value stored in the cell. An **alias** is another reference to the same object. This assignment gives each `dt_ref` one owned cell and an explicit released state.

An owned reference before and after release

Release frees the cell and leaves a handle whose flag rejects later access.

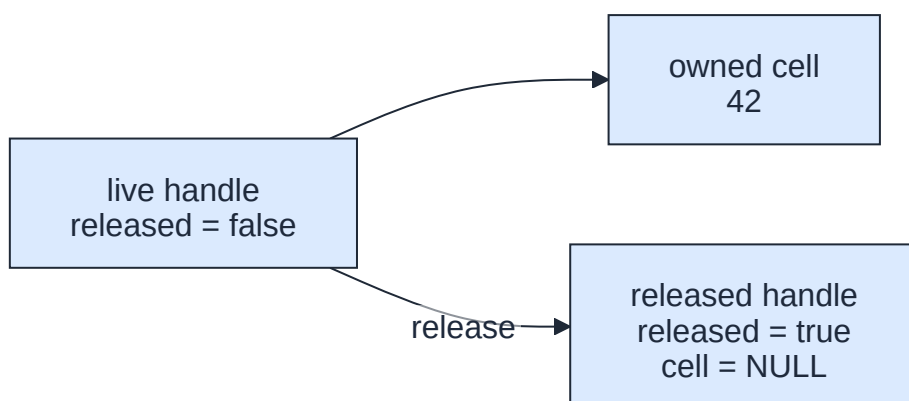
Record the lifetime state

Figure 10: Record the lifetime state

Start with `cases/ownership/ref_released.case`. The remaining ownership and post-release cases separate borrowing, aliasing, double release, and the final leak check.

```

1 dt_ref  *dt_ref_new(dt_value v);
2 dt_status dt_ref_borrow(const dt_ref *p, dt_value *out);
3 dt_status dt_ref_release(dt_ref *p);
4 bool     dt_ref_is_released(const dt_ref *p);
5 void     dt_ref_destroy(dt_ref *p);
  
```

Function	What it does	Fails with	Why that code
<code>dt_ref_new</code>	builds a reference owning a copy of <code>v</code> in a fresh cell	returns NULL	a constructor has no status to return
<code>dt_ref_borrow</code>	writes a copy of the cell's value to <code>*out</code>	<code>DT_ERR_RELEASED</code>	the cell is gone, so there's nothing to copy, and reading through the old pointer is the dangling read this module exists to catch
<code>dt_ref_release</code>	frees the cell and raises the released flag	<code>DT_ERR_RELEASED</code>	it was already released, and handing the same block back twice corrupts the allocator's own records
<code>dt_ref_is_released</code>	true once the reference has been released	cannot fail	it reads the flag, and the driver's leak sweep is what reads it
<code>dt_ref_destroy</code>	frees a surviving cell, then the handle	cannot fail	the driver calls it at exit, after any leak has already been reported

Three failures live here. C reports none of them for you.

A **dangling reference** still holds an address after its object was freed. Reading through it has undefined behavior. You might see the old value, some other allocation's bytes, or a terminated program. No outcome is promised.

A **double release** hands the same allocation to `free` twice. That also has undefined behavior. The visible failure can occur immediately or later.

An **unreleased allocation** still has an owner when the program reaches its final check. In this driver, the handle hasn't vanished. The driver can still reach it, report `DT_ERR_LEAK`, and then destroy the cell during cleanup. The memory remains reachable throughout that sequence.

A released flag lets this small interface refuse the first two operations with `DT_ERR_RELEASED`. The driver's final check uses the same flag to report the third. These checks enforce one exercise's explicit contract. They don't implement garbage collection or reference counting.

Ownership stops at the cell. `dt_ref_new` copies the value into storage the reference owns. `dt_ref_borrow` copies that stored `dt_value` into `*out`. The borrow leaves ownership unchanged. If the value points at a string, the environment still owns that string. Releasing the reference frees only its cell. One corpus case makes 2 references alias the same string, then releases both.

Follow one reference from birth to exit, with the flag doing all the work.

```

1  ref new p 42      dt_ref_new    ->  a cell holding 42, flag false
2  ref borrow p     dt_ref_borrow ->  DT_OK, 42, because the flag is false
3  ref release p    dt_ref_release ->  DT_OK, the cell is freed, flag true
4  ref borrow p     dt_ref_borrow ->  DT_ERR_RELEASED, because the flag is true
5  ref release p    dt_ref_release ->  DT_ERR_RELEASED, so nothing is freed twice
6  (exit)           the sweep     ->  the flag is true, so no leak is reported

```

Delete the release command and the final check finds the flag still false. It reports `DT_ERR_LEAK` before cleanup destroys the cell and handle. That's why `dt_ref_is_released` has to return the stored flag. A constant would lie on one of the two paths. The release function also sets the cell pointer to `NULL`, but the flag remains the guard that every later operation must check.

The Run Contract

Every case file is executed by the same front end. `./run <case-file>` hands the file to the driver, which reports its result through one of 3 exit codes. They're the same 3 codes the laboratory interpreter uses, with the same meanings.

```

1  ./run cases/normal/array_basics.case

```

Code	When
0	every command parsed and ran, and no reference was left unreleased
65	the case file was rejected before anything ran
70	a command ran and then faulted

The exit code is the whole verdict. Code 0 means every command ran and no reference was left unreleased. The leak sweep enforces that last part, so an unreleased reference turns a would-be 0 into a 70. Code 65 means the driver refused the file before anything ran. Code 70 means it ran the file and a command faulted partway through.

The driver is built to keep the 65 and 70 split honest. Before executing a single one, it reads the whole file and checks every command's verb, argument count, argument kinds, and identifiers. So a file with a typo on line 40 produces no output at all, even though lines 1 through 39 were perfectly good. The corpus has a case for that. It feeds two valid commands followed by an invalid one and expects empty stdout.

That's the same separation the compiler-structure unit of the syllabus describes: scanning, parsing, and execution finish one stage at a time, so a later stage never starts on a file an earlier stage rejected. A syntax error can't half-run your program.

Program output goes to stdout. Diagnostics go to stderr. The corpus compares stdout and the exit code, so your diagnostics exist for your benefit. A diagnostic that doesn't include a line number is one you'll regret around day five.

Running the Checks

```
1 ./build.sh      # one clean build
2 ./check.sh     # the complete public automated check
```

`check.sh` does three things: builds, runs the 63 published cases, then builds a second time with AddressSanitizer and UndefinedBehaviorSanitizer switched on and runs the same 63 again. Before any of that, its first run fetches the test runner, `run_tests.py`, from the shared `cmisc-124-harness` repository. It's the same runner the laboratory activities use, pinned to tag `v1.1`, and `check.sh` keeps it in your working tree for the runs after. That fetch needs network. If it can't reach the repository, `check.sh` prints `check.sh: could not fetch the harness. Check your network.` and exits 1.

Why the Second Run Exists

The first run compares text and exit codes. Plenty of C bugs can still produce the expected result. A read one element past an allocation might happen to find accessible memory. A use of freed memory might return the old bytes in one run. Both have undefined behavior, so today's plausible output proves very little.

The sanitizers catch that category. Writing this starter, AddressSanitizer found a leak in the driver you were given, on the malformed-input path, that every correctness check had passed straight through. Nothing about the output was wrong. The bug was invisible to a comparison of stdout. That's the whole reason the sanitized leg exists.

Windows Caveat

MinGW GCC ships neither `libasan` nor `libubsan`. On MSYS2 the sanitized build can't link at all, and `check.sh` says so and skips that leg:

```
1 SKIP: cc cannot link -fsanitize=address,undefined here.
2 SKIP: this is expected on MSYS2 and MinGW, which ship no libasan.
3 SKIP: the sanitized run happens on Linux, on macOS, and in CI.
```

The skip comes from the toolchain. Your repository's GitHub Actions workflow runs on Ubuntu and macOS, where both sanitizers work. Leak detection is a Linux-only extra. Linux AddressSanitizer turns it on by default, and Apple's ships no leak checker at all, so the Ubuntu job is where a leak gets caught. Push your work and read both jobs for the memory evidence the rubric uses. Don't leave that until the final push.

Everything Is Public

The 63 cases in `cases/` are the 63 cases I grade with. There's no hidden suite, no grader-only script, no extra check that appears at submission time. `check.sh` is the same file for you, for your CI, and for me.

When I grade, I replace `cases/` with the published copy before running anything. If you edited a case file to make it pass, you get the original back and it fails. If you *added* files, that's fine and they're ignored. The corpus is a fixed target on purpose, so that a passing run means the same thing for every pair. Your repository being a fork is what makes that check cheap: grading starts from the released starter commit and reads the fork's diff against it, so an edited fixture is visible as a diff in the open.

Milestones

10 modules, 10 days, one a day. Release day and deadline day aren't in the count, so they're your slack. Work together each day. If you each take five modules end to end, half the code will remain foreign to each of you. I will ask about it.

Day	Date	Module	Goes green
1	Sat Sep 26	<code>dt_value.c</code>	<code>normal/union_readers</code>
2	Sun Sep 27	<code>dt_int.c</code>	<code>normal/int_arithmetic</code>
3	Mon Sep 28	<code>dt_str.c</code>	<code>normal/string_building</code>
4	Tue Sep 29	<code>dt_enum.c</code>	<code>normal/enum_names</code>
5	Wed Sep 30	<code>dt_array.c</code>	<code>normal/array_basics</code>
6	Thu Oct 1	<code>dt_map.c</code>	<code>normal/map_basics</code>
7	Fri Oct 2	<code>dt_record.c</code>	<code>normal/record_basics</code>
8	Sat Oct 3	<code>dt_tuple.c</code>	<code>normal/tuple_basics</code>
9	Sun Oct 4	<code>dt_list.c</code>	<code>normal/list_basics</code> , <code>normal/nested_values</code>
10	Mon Oct 5	<code>dt_ref.c</code>	the whole corpus

Friday the 25th is for reading. Both of you go through this manual and `include/dt.h` end to end, run `./check.sh` once so you've seen the failure, and meet your partner. Nothing gets written that day.

Day 1 is `dt_value.c`, and only its 3 readers. It's the shortest work in the set. It establishes the tag discipline that the other nine modules assume, so do it together on one screen.

Three lecture slots fall inside the run. They're September 28, October 1, and October 5, and there's no meeting on any of them, so they're yours.

Day 6 is the one to protect. `dt_map.c` is roughly four times the code of `dt_enum.c`. It's the only module where the hash itself can cause a wrong answer. Start it Wednesday evening if you can, and do it together.

Day 9 is when `normal/nested_values` finally goes green. It needs array, map, record, tuple, and list all working at once. That's what proves the shared union works. Day 10 ends with the whole corpus green and pushed, with the sanitized run confirmed in Actions. Tuesday the 6th is the deadline. Use it to finish `ANALYSIS.md`, read the commit history without rewriting it, run one last check, and send the emails.

If you fall behind, use your current failures to decide what to finish next. Source dependencies are narrow, but test dependencies aren't. Only `dt_array` and `dt_record` call another assignment module. Both call `dt_value_nil`. The shared test paths still cross module boundaries. They include `nested_values`, `many_allocations`, tag checks, and string-backed values. One unfinished module can therefore fail a case whose filename points to another module.

Important Dates

Event	Date
Problem set released	Friday, September 25, 2026
Pairing sheet published	Friday, September 25, 2026
Submission deadline	Tuesday, October 6, 2026
First long exam	Wednesday, October 7, 2026

The first long exam covers Units 1 through 5 on Wednesday October 7. It falls outside the usual Monday and Thursday pattern so that every section can sit it together. Submit early and you may use the rest of the time to study.

Joint Analysis

Write `ANALYSIS.md` in your repository. Both of you write it, together, after the code works. It's 4 questions and a fifth of the grade, so treat it seriously.

1. Pick 3 of the 10 categories. For each, pick a language that gives it to you for free and say what that language pays for it. "Python has dictionaries" isn't an answer. What does Python's dictionary cost in memory or in speed compared to what you built, and where would you notice?
2. You wrote the tag check in `dt_value_as_int` by hand. Some languages don't let you. They make the tagged union a language construct, so the compiler writes the check for you, refuses to compile a read that skips it, and refuses to compile a set of cases that misses one. Rust's `enum` and `match` work this way, and so do ML's datatypes and Swift's enumerations with associated values. What does the C version let you do that a compiler enforcing the check wouldn't, and is any of it worth wanting?
3. Your `dt_map` keeps insertion order separately from the hash buckets, which is memory spent on something no lookup uses. Argue the other side: describe a design that drops it, say what breaks, and say whether you'd ship it.
4. Compare access after release with an allocation that remains unreleased at the driver's final check. What damage can each cause in a long-running server? How does that answer change for a command-line tool that exits in a second?

Deliverables

A public GitHub repository for the pair, containing only these changes:

1. The 10 `src/dt_*.c` files, implemented
2. `ANALYSIS.md`, with your joint answers
3. The Pair section in `README.md`, with every member's full name and GitHub username

Keep everything else unedited, including `include/dt.h`, the 4 given source files, `cases/`, and `.github/workflows/test.yml`. The workflow must be green on the commit you submit.

Both members commit. The commit history is read. A repository where one author appears 60 times and the other appears twice is a repository where one person did the work. That's visible from the graph and it affects both grades, in opposite directions. Collaboration is the 15% row of the rubric below, and a history like that puts the pair in its bottom band, so the member who wrote everything loses marks too.

Use the semantic commit prefixes from the syllabus. An **atomic commit** records one focused working change, so its message and diff tell the same story. The prefixes are listed under Individual Grading, with an example for each: `docs`, `feat`, `refactor`, `fix`, `ux`, `meta`, and `test`. `feat: implement dt_array bounds check` tells me something. `update` doesn't.

After the final push, open the repository's commit history on GitHub and copy the full hash of the commit you want graded. One designated member emails both items included in the body in this form:

```
1 Repository: https://github.com/<your-account>/cmsc-124-ps1-starter
2 Commit: <full commit hash copied from GitHub>
```

The submitted hash pins the version I grade. The green workflow run must belong to that commit. A later commit doesn't change the submitted version.

CC the rest of your group. Adhere to the following subject line, joining every group member with & in the same LastName, Initials format: [CMSC 124 Lec] PS1 Group: LastName1, Initials1 & LastName2, Initials2. A trio adds a third person the same way.

Then each member emails a short `reflection.txt` individually. Don't add this file to the repository. Say which modules you wrote and which your partner explained to you. Include the challenges you met and what you learned. Use this subject line: [CMSC 124 Lec] PS1: LastName, Initials.

For example: [CMSC 124 Lec] PS1 Group: Sanchez, SM & Jocsing, RA and [CMSC 124 Lec] PS1: Sanchez, SM.

Common Pitfalls

- Testing for overflow after the arithmetic. The arithmetic already has undefined behavior. An optimized build can transform or delete the check. `build.sh` builds Release at `-O3`.
- `strlen` or `strcmp` on your `dt_str` bytes. Both stop at a zero byte. One of the cases has a zero byte in the middle on purpose.
- A bounds check with only an upper end. A below-bound index can read outside the allocation, which has undefined behavior.
- Computing `index - lower_bound` in signed arithmetic before you know the difference fits. `LLONG_MAX - LLONG_MIN` can't fit in `long long`.
- Two copies of the bounds check, one in `get` and one in `set`, which agree today.
- `dt_list_free` following the tail. It frees cells other lists still reach. The sanitizer will tell you so in CI even though the corpus passes locally.
- Returning `nil` for a missing key or an empty list's car. Absent and `nil` are different answers. The corpus checks that they stay different.
- Freeing the values inside a container. The environment owns those and frees them too. This is a double free. It's the one most likely to look fine on Windows and fail in Actions.
- Reading 33/63 on a fresh clone as halfway. It's zero.
- Editing a case file to make it pass. Don't. The corpus is replaced from the published copy before grading.

Academic Honesty

Submitting code that a large language model generated, which you didn't write, can't read, and can't explain, is cheating. It's subject to failure in the course and to harsh disciplinary action. Using a model to explain a concept, or to help you read an error message, isn't what this is about. The line is whether the code in your repository is code you can account for.

The 10 modules have to agree about ownership, status codes, and absent values. Generated code tends to disagree across file boundaries. `ANALYSIS.md` asks you to defend decisions visible in your own source. I'll ask either of you about any line in the repository, when we see each other, without warning. Code you didn't write is hard to talk about for ten seconds, let alone 2 minutes.

Grading Rubric

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Implementation Correctness (50%)	<code>check.sh</code> reports all 63 published cases passing. The code handles each boundary because it was written to rather than tuned until the case went green.	Every <code>normal/</code> case passes with a few boundary or capacity cases failing.	The common paths work; several boundary, tag, or ownership cases fail.	<code>normal/</code> cases still failing, or the build is broken.
Memory Discipline (15%)	Both the Ubuntu and macOS CI jobs complete the sanitized corpus: no double free, out-of-bounds access, or undefined operation, and no leak on the Ubuntu job.	One sanitizer finding that does not change public output.	Several findings, or a leak on a common path.	A sanitized job crashes, or either CI job never ran.
Joint Analysis (20%)	<code>ANALYSIS.md</code> answers all four questions with named languages, real costs, and positions argued from the implementation.	Solid answers, thinner on the tradeoffs.	Restates the manual; the alternatives aren't really weighed.	Missing, or plainly written by one person the night before submission.
Collaboration (15%)	The Git history contains atomic semantic commits from both members across the work period. Each member implements and reviews modules.	Both members contribute, with uneven distribution or vague messages.	Contributions are lopsided or crammed into the last day.	One member did the work, or the pair rewrote the history.